

Dokumentation

API-Dokumentation

Projektplanung

Kendo DBI

#dbi

1 Team Projektidee

team

- Luis
- Luis
- Daniel

Projektidee

Kurzbeschreibung

ken ~ the range of perception, understanding, or knowledge

Kendo ist eine Plattform um shader zu schreiben und zu teilen, basically [shadertoy](#). Kendo soll außerdem übliche socialmediafeaturers wie z.B. liken, kommentieren und selber posten beinhalten.

Domäne

unsere Dömäne soll user verwalten außerdem soll sie unsere shader samt shaderpropertys und logging speichern.

Überlegungen Enteties und Beziehungen

haves

Must have

- User
 - Post
 - Get
 - Get All
 - Get by Id
 - Get by name
 - Patch
 - Delete
- Comment
 - Post
 - Get All of shader
 - Delete
- Shader
 - Get all
 - Get all by user
 - Get by id
 - Post
 - Patch
 - Delete
- Tags
 - Post
 - Get
 - Get by id
 - Get by shader
- Logging
 - Post Log
 - Get Log

Nice to haves

- likes
- follows

Aufgabenverteilung

Zuständigkeiten

Luis K

- rendering
- code Editor

Luis B

- Kommentarsystem
- Filter
- Tag system

Daniel W

- User Handling
- password hashing

2. Planungsphase

Normalformen

Shader

ShaderID	PK	INTEGER
Code		TEXT
<i>UserID</i>	FK	

User

UserID		INTEGER
Name		VARCHAR (50)
Passwd		TEXT

ShaderTags

<i>ShaderID</i>	FK	
<i>TagsID</i>	FK	

Tags

TagsID	INTEGER
Text	VARCHAR(32)

Comments

CommentsID	PK	INTEGER
Message		VARCHAR(256)
<i>ShaderID</i>	FK	
<i>UserID</i>	FK	

Log

LogID	PK	INTEGER
TEXT		TEXT
Loglvl		INTEGER
<i>UserID</i>	FK	

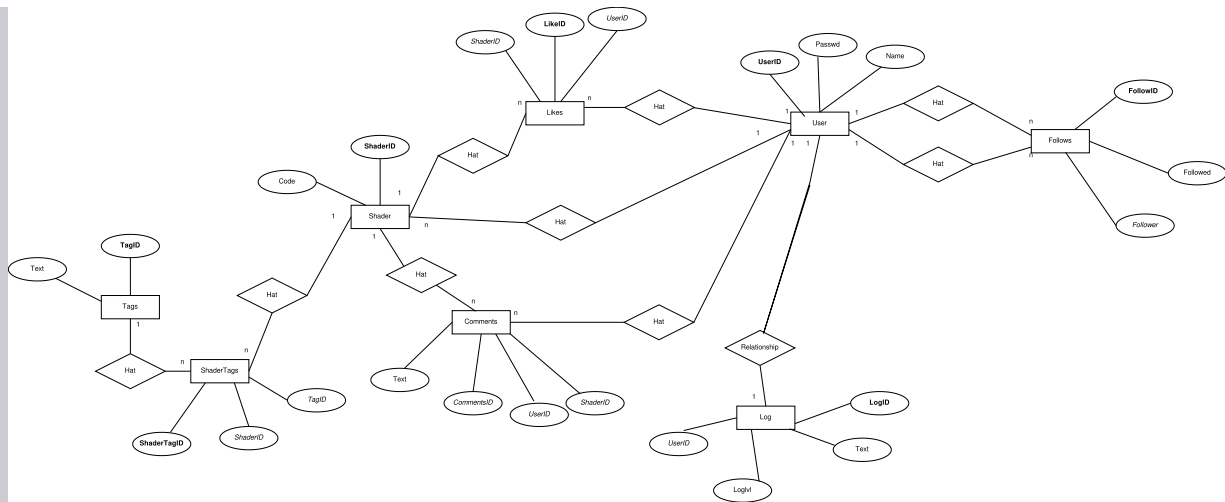
Follows

FollowID	PK	INTEGER
<i>Follower</i>	FK : UserID	
<i>Followed</i>	FK : UserID	

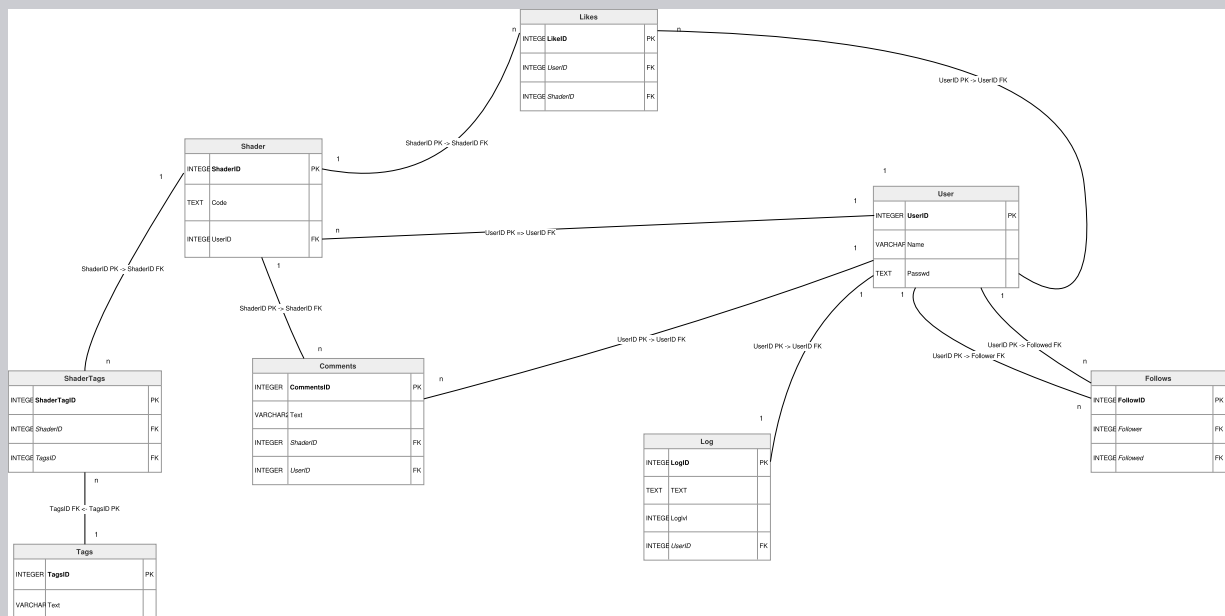
Likes

LikeID	PK	INTEGER
<i>UserID</i>	FK	
<i>ShaderID</i>	FK	

ERM



RM



Git repo

https://github.com/iSketchup/KENDO_DB/

Inhaltsverzeichnis

- [Projektlaufzeit](#)
- [Allgemeine Informationen](#)
 - [Basis-URL](#)
 - [Authentifizierung](#)
 - [Allgemeine JSON-Regeln](#)
 - [Standard-Fehler bei Validierung](#)
- [Root-Endpunkt](#)
- [User-API Dokumentation](#)
 - [GET /user/{username}](#)

- [POST /user/login](#)
- [POST /user/](#)
- [PUT /user/](#)
- [DELETE /user/](#)
- [POST /user/Log](#)
- [Admin-API Dokumentation](#)
 - [GET /admin/Log](#)
 - [DELETE /admin/Log/{user_id}](#)
 - [GET /admin/](#)
 - [GET /admin/{username}](#)
 - [POST /admin/login](#)
 - [POST /admin/](#)
 - [PUT /admin/](#)
 - [DELETE /admin/](#)
- [Shader-API Dokumentation](#)
 - [GET /{user_id}/shaders/](#)
 - [GET /{user_id}/shaders/{shader_id}](#)
 - [GET /{user_id}/shaders/filter/](#)
 - [POST /{user_id}/shaders/new](#)
 - [PUT /{user_id}/shaders/{shader_id}](#)
- [ShaderTag-API Dokumentation](#)
 - [GET /{user_id}/shaders/shadertag/{shader_id}](#)
 - [POST /{user_id}/shaders/shadertag/{shader_id}/{tag_id}](#)
 - [DELETE /{user_id}/shaders/shadertag/{shader_id}/{tag_name}](#)
- [ShaderTexture-API Dokumentation](#)
 - [GET /{user_id}/shaders/{shader_id}/shadertexture](#)
 - [POST /{user_id}/shaders/{shader_id}/shadertexture](#)
 - [PUT /{user_id}/shaders/{shader_id}/shadertexture/{TextureId}](#)
 - [DELETE /{user_id}/shaders/{shader_id}/shadertexture/{TextureId}](#)
- [Tags-API Dokumentation](#)
 - [GET /tags/](#)
 - [GET /tags/{tag_id}](#)
 - [POST /tags/](#)
 - [DELETE /tags/delete_id/{tag_id}](#)
 - [DELETE /tags/delete_name/{tag_name}](#)
- [Likes-API Dokumentation](#)
 - [GET /{user_id}/{shader_id}/likes/](#)
 - [POST /{user_id}/{shader_id}/likes/](#)
 - [DELETE /{user_id}/{shader_id}/likes/](#)

- [Comments-API Dokumentation](#)
 - [GET /{user_id}/{shader_id}/comments/](#)
 - [POST /{user_id}/{shader_id}/comments/](#)
 - [DELETE /{user_id}/{shader_id}/comments/{comment_id}](#)
- [Datenbankmodell](#)

Instruction

- Repo Clonen
- .env File erstellen mit folgendem Inhalt: `KENDO_KEY=<dein_API_key>`
- requirements installieren
- init_db.py File runnen lassen um die Datenbank mit test Daten zu befüllen
- main.py File runnen oder in /src uvicorn.app
- auf diese URL gehen <https://127.0.0.1:8000/docs>

Projektlaufzeit

Datum	Autor	Aufgabe
2026-06-18 20:29:43	jnhghgi	init_db now fills your db with test data
2026-06-18 15:13:28	iSketch_up	Merge remote-tracking branch 'origin/main'
2026-06-18 15:13:16	iSketch_up	wip
2026-06-18 12:44:55	jnhghgi	added shader Author
2026-06-18 11:17:21	jnhghgi	imporved shader tags structur
2026-06-18 10:29:00	iSketch_up	Merge remote-tracking branch 'origin/main'
2026-06-18 10:28:35	iSketch_up	added a respons code to base root for checking availability
2026-06-18 08:09:32	jnhghgi	imporved shader tags structur
2026-06-17 21:59:47	jnhghgi	fixed like_create not toggling between creating and deleting like
2026-06-17 21:36:41	jnhghgi	fixed wrong response for get_by_filters

Datum	Autor	Aufgabe
2026-06-17 20:47:57	Daniel WALSER	Fehlermeldung ist jetzt im user und Admin für die Suche nach dem Namen vorhanden.
2026-06-17 20:14:24	Daniel WALSER	Bug fix. Im Login wird jetzt genauer geprüft, ob das Passwort vorhanden ist.
2026-06-17 20:00:39	Daniel WALSER	Bug fix.
2026-06-17 13:13:47	Daniel WALSER	Server ip anpassung.
2026-06-17 10:09:38	Daniel WALSER	Anpassung vom Admin.py. Hashing ist auf der Server Seite möglich.
2026-06-17 06:58:05	jnhghgi	Merge remote-tracking branch 'origin/main'
2026-06-17 06:57:31	jnhghgi	improved error handling for tag and comments
2026-06-17 07:49:28	Daniel WALSER	Bug Fix. Versuch das SSL sicher zu machen. Im Backend ist es möglich ein hashpassword zu erzeugen.
2026-06-16 19:25:22	jnhghgi	created init_db.py
2026-06-16 21:14:22	Daniel WALSER	Bug fix.
2026-06-16 20:59:13	Daniel WALSER	Bug fix.
2026-06-16 20:06:46	Daniel WALSER	is_admin attribut hinzugefügt. Prüfung dass nur ein Admin existieren darf.
2026-06-16 17:34:21	jnhghgi	changed the comment route to return serialized comments
2026-06-16 18:58:12	Daniel WALSER	is_admin attribut wurde dem userResponse auch hinzugefügt.
2026-06-16 18:51:40	iSketch_up	added a route for creating a new shader loaded with a sample
2026-06-16 18:23:34	Daniel WALSER	ADmin wurde somit auch erstellt.
2026-06-15 19:07:05	iSketchup	typos
2026-06-15 18:40:22	iSketchup	Merge remote-tracking branch 'origin/main'
2026-06-15 18:39:40	iSketchup	gitignore update
2026-06-15 18:06:04	Daniel WALSER	Requirements angepasst

Datum	Autor	Aufgabe
2026-06-15 15:21:48	jnhghgi	changed get_filter_by_user_id to a more flexible filter
2026-06-15 12:49:48	Daniel WALSER	API-KEY wurde noch fertig implementiert.
2026-06-14 17:12:00	iSketchup	reworks + textures added
2026-06-14 13:06:06	jnhghgi	Merge remote-tracking branch 'origin/main'
2026-06-14 15:00:12	iSketch_up	holy wip
2026-06-14 12:51:16	jnhghgi	tryed to implement dry
2026-06-11 12:30:10	Daniel WALSER	Update: Man kann eigentlich auch nur den Usernamen ohne Passwort abändern, anstatt nur das Passwort
2026-06-11 12:28:17	Daniel WALSER	Jetzt kann für User auch nur das Passwort öndern
2026-06-09 22:42:48	iSketchup	reworks + textures added
2026-06-09 15:54:04	jnhghgi	fixed get shader by user
2026-06-09 15:45:48	jnhghgi	deleted unused router_per_user, updated shader router, WIP shader per user_id
2026-06-09 15:43:10	jnhghgi	fixed shadertag models and tag,like route structure
2026-06-08 15:28:13	Daniel WALSER	Update: Da es Probleme gab, gibt es eine Datei die das SSL erzeugt.
2026-06-08 15:08:28	Daniel WALSER	SSL-Zertifikat wird jetzt erstellt, wenn man keines zum starten hat.
2026-06-08 14:50:11	Daniel WALSER	Neue Route um die User nur nach dem Namen finden kann.
2026-06-08 13:59:51	iSketch_up	added a Texture model and a Name Column for Shaders
2026-06-08 13:59:24	iSketch_up	gitignore update
2026-06-07 15:53:06	Daniel WALSER	Anpassung von ChangeUser und DeleteUser an das C# Programm.
2026-06-06 23:37:43	jnhghgi	updated route structures
2026-06-06 16:13:06	iSketchup	wip

Datum	Autor	Aufgabe
2026-06-06 13:43:45	jnhghgi	deleted shadertags route
2026-06-06 13:38:20	jnhghgi	Created tags and shadertags and a simple auth not implemented yet tho
2026-06-05 20:24:08	iSketchup	updated erm and rm
2026-06-05 18:28:56	Daniel WALSER	Ohne Zertifikat kann man den Server auch starten.
2026-06-05 17:55:40	Daniel WALSER	Hier ist noch das ERM u. RM
2026-06-05 17:29:03	Daniel WALSER	Hier wurde HTTPS eingebunden.
2026-06-03 16:59:05	iSketch_up	Merge remote-tracking branch 'origin/main'
2026-06-03 16:58:50	iSketch_up	renamed
2026-06-03 11:50:47	Daniel WALSER	ERM-RM angepasst
2026-06-01 12:09:12	Daniel WALSER	Requirements sind auf aktuellen Stand
2026-05-31 20:52:01	iSketchup	shader can be updated
2026-05-31 20:42:25	iSketchup	reworked how shaders work:)
2026-05-29 09:51:29	Daniel WALSER	3. Teil Login
2026-05-28 16:44:26	Daniel WALSER	Versuch 2.teil: Login ermöglichen.
2026-05-27 15:08:24	Daniel WALSER	Versuch: Das Login ermöglichen.
2026-05-27 13:13:59	Daniel WALSER	Hier wurden mal die Hashing Methoden erstellt.
2026-05-25 14:18:04	jnhghgi	Created comments router and updated likes router
2026-05-25 13:54:38	jnhghgi	Fixed like router not being able to find foreign key relationships
2026-05-25 12:49:09	Daniel WALSER	User: Eine Alias wurde eingefügt, damit der Server einen neuen User erstellen kann.
2026-05-24 18:39:53	Daniel WALSER	Merge remote-tracking branch 'origin/main'

Datum	Autor	Aufgabe
2026-05-24 18:39:43	Daniel WALSER	PUT: Username abändern sodass zwei den gleichen haben ist nicht möglich.
2026-05-24 16:33:05	jnhghgi	Created likes router and fixed likes model
2026-05-24 18:32:06	Daniel WALSER	POST: User mit gleichen Namen sind nicht mehr möglich.
2026-05-24 17:14:13	iSketchup	shaders are postable
2026-05-24 17:11:20	iSketchup	Merge remote-tracking branch 'origin/main'
2026-05-24 17:11:03	iSketchup	made rooute for getting shaders by id
2026-05-24 17:07:49	Daniel WALSER	Merge remote-tracking branch 'origin/main'
2026-05-24 17:07:28	Daniel WALSER	Post: gleiche Usernamen sind nicht mehr erlaubt.
2026-05-24 17:06:19	iSketchup	made main include shader router
2026-05-24 17:06:11	Daniel WALSER	Put Methode hinzugefügt und das Delete wurde gefixt.
2026-05-24 16:51:32	Daniel WALSER	Put Methode hinzugefügt und das Delete wurde gefixt.
2026-05-23 09:24:42	Daniel WALSER	Maximale Länge von Strings angepasst. Routen angepasst.
2026-05-22 23:10:01	jnhghgi	Created models for Tag, ShaderTag, Comment, Shader and improved User model
2026-05-23 00:35:16	iSketchup	cleanup
2026-05-23 00:33:42	iSketchup	updated gitignore
2026-05-21 14:04:40	Daniel WALSER	Hier ist die FastAPI für unser Projekt. Der User wurde somit auch erstellt.
2026-05-21 10:08:31	jnhghgi	Create Boilerplate Code for FastAPI
2026-05-21 10:06:36	jnhghgi	init
2026-05-18 22:13:26	iSketchup	init
2026-05-18 22:12:57	iSketchup	init

Datum	Autor	Aufgabe
2026-05-18 18:18:10	iSketchup	init
2026-05-18 18:17:22	iSketch_up	init
2026-05-18 18:04:08	iSketchup	Initial commit

Allgemeine Informationen

Basis-URL

Beim lokalen Start über `src/main.py` wird der Server auf Port `8000` gestartet. Wenn SSL-Zertifikate vorhanden sind oder automatisch erzeugt werden konnten, läuft die API lokal über:

```
https://127.0.0.1:8000
```

In den Beispielen wird allgemein diese Schreibweise verwendet:

```
http(s)://<server>
```

`<server>` steht dabei für die IP-Adresse oder Domain des Servers.

Authentifizierung

Für User- und Admin-Endpunkte wird ein API-Key im Header erwartet.

Header	Wert
<code>X-API-KEY</code>	Wert aus der Umgebungsvariable <code>KENDO_KEY</code>

Beispiel:

```
X-API-KEY: mein_api_key
```

Wenn der API-Key fehlt oder falsch ist, antwortet das Backend mit:

```
{
  "detail": "Invalid API Key"
```

```
}
```

Code	Bedeutung
401	API-Key fehlt oder ist ungültig

Hinweis: Im aktuellen Code ist die API-Key-Prüfung bei `User` und `Admin` aktiv. Bei `Shader`, `Tags`, `Likes` und `Comments` ist keine aktive API-Key-Abhängigkeit eingetragen.

Allgemeine JSON-Regeln

Alle Request-Bodys und Responses werden als JSON übertragen.
IDs sind Integer-Werte. Texte werden als String übertragen.

Wichtige Längenregeln:

Feld	Regel
<code>UserName</code>	1–31 Zeichen, keine Leerzeichen
<code>AdminName</code> / <code>UserName</code>	1–31 Zeichen, keine Leerzeichen
<code>ShaderName</code>	maximal 63 Zeichen in der Datenbank
<code>TagName</code>	maximal 31 Zeichen
<code>CommentText</code>	maximal 511 Zeichen laut Pydantic-Modell
<code>Texture64</code>	Base64-codierte Textur als String

Standard-Fehler bei Validierung

Das Backend hat einen eigenen Validation-Error-Handler. Bei ungültigen Request-Daten sieht die Antwort so aus:

```
{
  "status": "validation_error ",
  "errors": [
    {
      "field": "UserName",
      "msg": "Field required"
    }
  ]
}
```

Code	Bedeutung
404	Validierungsfehler im Request

Root-Endpunkt

GET /

Prüft, ob die API erreichbar ist. Der Endpunkt gibt eine einfache Statusmeldung zurück.

Beispiel-Anfrage:

```
GET /
```

Response 200:

```
{
  "success": true,
  "status_code": 200,
  "message": "Hello World\nBesuche /docs für die API"
}
```

User-API Dokumentation

Basis-URL: `http(s)://<server>/user`

Authentifizierung: Jeder Request braucht einen gültigen API-Key im Header `X-API-KEY`.

Wichtige Regeln vorab

Der `UserName` darf keine Leerzeichen enthalten und muss 1–31 Zeichen lang sein.

Passwörter können als Klartext oder bereits als bcrypt-Hash (`$2a$`, `$2b$`, `$2y$`, exakt 60 Zeichen) geschickt werden. Das Backend erkennt das beim Erstellen eines Users selbst und hasht Klartextpasswörter automatisch.

GET /user/{username}

Gibt die Daten eines bestimmten Benutzers zurück.

URL-Parameter:

Parameter	Typ	Pflicht	Beschreibung
username	string	ja	Name des Benutzers

Beispiel-Anfrage:

```
GET /user/MaxMustermann
```

Response 200:

```
{
  "UserName": "MaxMustermann",
  "passwd": "$2b$12$...",
  "UserId": 1,
  "is_admin": false
}
```

Fehler:

Code	Grund
404	Benutzer wurde nicht gefunden
401	API-Key fehlt oder ist falsch

POST /user/login

Prüft Benutzername und Passwort. Bei Erfolg werden die Benutzerdaten zurückgegeben.

Request-Body:

```
{
  "UserName": "MaxMustermann",
  "passwd": "meinPasswort123"
}
```

Feld	Typ	Pflicht	Beschreibung
UserName	string	ja	Benutzername
passwd	string	ja	Passwort im Klartext

Hinweis: Wenn das gespeicherte Passwort noch kein bcrypt-Hash ist, wird es nach einem erfolgreichen Login automatisch gehasht und gespeichert.

Response 200:

```
{
  "UserName": "MaxMustermann",
  "passwd": "$2b$12$...",
  "UserId": 1,
  "is_admin": false
}
```

Fehler:

Code	Grund
404	Benutzer nicht gefunden
401	Passwort falsch oder Login ungültig

POST /user/

Legt einen neuen Benutzer an.

Request-Body:

```
{
  "UserName": "NeuerUser",
  "passwd": "sicheresPasswort"
}
```

Feld	Typ	Pflicht	Regeln
UserName	string	ja	1–31 Zeichen, keine Leerzeichen
passwd	string	ja	Klartext oder bcrypt-Hash

Response 200:

```
{
  "UserName": "NeuerUser",
  "passwd": "$2b$12$...",
  "UserId": 5,
}
```



```
"is_admin": false
}
```

Fehler:

Code	Grund
409	Ein Benutzer mit diesem Namen existiert bereits
401	API-Key fehlt oder ist falsch

PUT /user/

Ändert Benutzernamen und/oder Passwort eines bestehenden Benutzers.

Query-Parameter:

Parameter	Typ	Pflicht	Beschreibung
username	string	ja	Aktueller Name des Benutzers

Request-Body:

```
{
  "UserName": "NeuerName",
  "passwd": "neuesPasswort"
}
```

Beispiel-Anfrage:

```
PUT /user/?username=AlterName
```

Response 200:

```
{
  "UserName": "NeuerName",
  "passwd": "neuesPasswort",
  "UserId": 5,
  "is_admin": false
}
```

Fehler:

Code	Grund
404	Benutzer mit dem alten Namen nicht gefunden
409	Der neue Benutzername ist bereits vergeben
401	API-Key fehlt oder ist falsch

DELETE /user/

Löscht einen Benutzer anhand seines Namens.

Query-Parameter:

Parameter	Typ	Pflicht	Beschreibung
username	string	ja	Name des zu löschenden Benutzers

Beispiel-Anfrage:

```
DELETE /user/?username=MaxMustermann
```

Response: 204 No Content

Fehler:

Code	Grund
404	Benutzer wurde nicht gefunden
401	API-Key fehlt oder ist falsch

POST /user/Log

Erstellt einen Log-Eintrag für einen User.

Request-Body:

```
{
  "LogText": "Shader wurde geöffnet",
  "user_id": 1
}
```

Feld	Typ	Pflicht	Beschreibung
LogText	string	ja	Text des Log-Eintrags
user_id	int	ja	ID des Benutzers

Response 200:

```
{
  "LogText": "Shader wurde geöffnet",
  "user_id": 1
}
```

Fehler:

Code	Grund
400	user_id existiert nicht in der User-Tabelle
401	API-Key fehlt oder ist falsch

Kurzübersicht User

Methode	Pfad	Beschreibung
GET	/user/{username}	Benutzer abrufen
POST	/user/login	Einloggen
POST	/user/	Benutzer erstellen
PUT	/user/	Benutzer bearbeiten
DELETE	/user/	Benutzer löschen
POST	/user/Log	Log-Eintrag erstellen

Admin-API Dokumentation

Basis-URL: http(s)://<server>/admin

Authentifizierung: Jeder Request braucht einen gültigen API-Key im Header X-API-KEY.

Wichtige Regeln vorab

Der Admin wird technisch in derselben Tabelle wie normale Benutzer gespeichert.

Ein Admin ist ein User mit `is_admin: true`.

Der `AdminName` wird im JSON über den Alias `UserName` verarbeitet. Es darf nur einen Administrator im System geben. Sobald ein Admin existiert, wird das Erstellen eines zweiten Admins abgelehnt.

GET /admin/Log

Gibt alle Log-Einträge zurück.

Beispiel-Anfrage:

```
GET /admin/Log
```

Response 200:

```
[
  {
    "LogText": "Shader wurde geöffnet",
    "user_id": 1
  },
  {
    "LogText": "User hat sich eingeloggt",
    "user_id": 2
  }
]
```

Fehler:

Code	Grund
401	API-Key fehlt oder ist falsch

DELETE /admin/Log/{user_id}

Löscht alle Log-Einträge eines bestimmten Users.

URL-Parameter:

Parameter	Typ	Pflicht	Beschreibung
user_id	int	ja	ID des Users, dessen Logs gelöscht werden

Beispiel-Anfrage:

```
DELETE /admin/Log/1
```

Response: 204 No Content

GET /admin/

Gibt eine Liste aller Benutzer im System zurück.

Beispiel-Anfrage:

```
GET /admin/
```

Response 200:

```
[
  {
    "UserName": "adminUser",
    "passwd": "$2b$12$...",
    "UserId": 1,
    "is_admin": true
  },
  {
    "UserName": "hansmüller",
    "passwd": "$2b$12$...",
    "UserId": 2,
    "is_admin": false
  }
]
```

GET /admin/{username}

Liefert die Daten zu einem bestimmten Benutzer.

URL-Parameter:

Parameter	Typ	Pflicht	Beschreibung
username	string	ja	Name des gesuchten Benutzers

Beispiel-Anfrage:

```
GET /admin/hansmüller
```

Response 200:

```
{
  "UserName": "hansmüller",
  "passwd": "$2b$12$...",
  "UserId": 2,
  "is_admin": false
}
```

Fehler:

Code	Grund
404	Kein Benutzer mit diesem Namen gefunden

POST /admin/login

Prüft die Login-Daten eines Admins.

Request-Body:

```
{
  "AdminName": "adminUser",
  "passwd": "adminPasswort"
}
```

Feld	Typ	Pflicht	Beschreibung
AdminName	string	ja	Name des Admin-Accounts
passwd	string	ja	Passwort im Klartext

Response 200:

```
{
  "UserName": "adminUser",
  "passwd": "$2b$12$...",
  "UserId": 1,
  "is_admin": true
}
```

Fehler:

Code	Grund
404	Benutzer nicht gefunden
401	Passwort falsch

POST /admin/

Legt den Administrator-Account an. Dieser Endpunkt kann nur erfolgreich sein, wenn noch kein Admin existiert.

Request-Body:

```
{
  "UserName": "adminUser",
  "passwd": "sicheresPasswort"
}
```

Feld	Typ	Pflicht	Regeln
UserName	string	ja	1–31 Zeichen, keine Leerzeichen
passwd	string	ja	Klartext oder bcrypt-Hash

Der neu angelegte Account bekommt automatisch:

```
"is_admin": true
```

Response 200:

```
{
  "UserName": "adminUser",
  "passwd": "$2b$12$...",
  "UserId": 1,
}
```

```
"is_admin": true
}
```

Fehler:

Code	Grund
409	Ein Benutzer mit diesem Namen existiert bereits
400	Es existiert bereits ein Administrator
401	API-Key fehlt oder ist falsch

PUT /admin/

Ändert Benutzername und/oder Passwort eines Benutzers.

Query-Parameter:

Parameter	Typ	Pflicht	Beschreibung
username	string	ja	Aktueller Name des Benutzers

Request-Body:

```
{
  "UserName": "neuerAdminName",
  "passwd": "neuesPasswort"
}
```

Beispiel-Anfrage:

```
PUT /admin/?username=adminUser
```

Response 200:

```
{
  "UserName": "neuerAdminName",
  "passwd": "neuesPasswort",
  "UserId": 1,
  "is_admin": true
}
```

Fehler:

Code	Grund
404	Benutzer nicht gefunden
409	Der neue Name ist schon vergeben

DELETE /admin/

Löscht einen Benutzer.

Query-Parameter:

Parameter	Typ	Pflicht	Beschreibung
username	string	ja	Name des zu löschenden Benutzers

Beispiel-Anfrage:

```
DELETE /admin/?username=hansmüller
```

Response: 204 No Content

Fehler:

Code	Grund
404	Benutzer nicht gefunden

Kurzübersicht Admin

Methode	Pfad	Beschreibung
GET	/admin/Log	Alle Logs abrufen
DELETE	/admin/Log/{user_id}	Logs eines Users löschen
GET	/admin/	Alle Benutzer abrufen
GET	/admin/{username}	Einzelnen Benutzer abrufen
POST	/admin/login	Admin-Login
POST	/admin/	Admin-Account erstellen
PUT	/admin/	Benutzer bearbeiten

Methode	Pfad	Beschreibung
DELETE	/admin/	Benutzer löschen

Shader-API Dokumentation

Basis-URL: `http(s)://<server>/{user_id}/shaders`

Authentifizierung: Im aktuellen Router ist kein API-Key aktiv.

Wichtige Regeln vorab

Ein Shader gehört immer zu einem User. Deshalb steht `user_id` im Pfad.

Bei Listen- und Einzelabfragen wird diese `user_id` auch verwendet, um anzugeben, ob genau dieser User den Shader geliked hat.

Ein vollständiger Shader enthält neben Code und Name auch Autor, Tags, Likes, Texturen und bei Einzelabfragen zusätzlich Kommentare.

Shader-Response

Die normale Shader-Antwort hat diese Struktur:

```
{
  "ShaderName": "Rainbow Diagonal",
  "ShaderCode": "#version 330 core\n...",
  "user_id": 2,
  "ShaderId": 1,
  "ShaderAuthor": "sebas",
  "ShaderTags": ["retro", "abstract"],
  "ShaderLikes": {
    "amount": 2,
    "liked_by_u": true
  },
  "ShaderTextures": [
    {
      "id": 1,
      "Texture64": "base64..."
    }
  ]
}
```

Bei einem einzelnen Shader kommt zusätzlich `ShaderComments` dazu:

```
"ShaderComments": [  
  {  
    "CommentAuthor": "guest",  
    "CommentText": "Looks amazing!"  
  }  
]
```

GET `/ {user_id} / shaders /`

Gibt alle Shader zurück.

URL-Parameter:

Parameter	Typ	Pflicht	Beschreibung
<code>user_id</code>	int	ja	Aktueller User, damit <code>liked_by_u</code> berechnet werden kann

Beispiel-Anfrage:

```
GET /1/shaders/
```

Response 200:

```
[  
  {  
    "ShaderName": "Rainbow Diagonal",  
    "ShaderCode": "#version 330 core\n...",  
    "user_id": 2,  
    "ShaderId": 1,  
    "ShaderAuthor": "sebas",  
    "ShaderTags": ["retro", "abstract"],  
    "ShaderLikes": {  
      "amount": 2,  
      "liked_by_u": true  
    },  
    "ShaderTextures": []  
  }  
]
```

GET `/ {user_id} / shaders / {shader_id}`

Gibt einen einzelnen Shader inklusive Kommentare zurück.

URL-Parameter:

Parameter	Typ	Pflicht	Beschreibung
user_id	int	ja	Aktueller User
shader_id	int	ja	ID des gesuchten Shaders

Beispiel-Anfrage:

```
GET /1/shaders/3
```

Response 200:

```
{
  "ShaderName": "Inverted Texture",
  "ShaderCode": "#version 330 core\n...",
  "user_id": 2,
  "ShaderId": 3,
  "ShaderAuthor": "guest",
  "ShaderTags": ["abstract", "raymarching"],
  "ShaderLikes": {
    "amount": 1,
    "liked_by_u": false
  },
  "ShaderTextures": [],
  "ShaderComments": [
    {
      "CommentAuthor": "sebas",
      "CommentText": "Cool inversion effect."
    }
  ]
}
```

Fehler:

Code	Grund
404	Shader wurde nicht gefunden

GET /{user_id}/shaders/filter/

Filtert Shader nach Autorname und/oder Shadername. Der Parameter `tags` ist im Code vorbereitet, aber die Filterlogik ist aktuell noch nicht aktiv.

Query-Parameter:

Parameter	Typ	Pflicht	Beschreibung
<code>shader_user_name</code>	string	nein	Filtert nach Autor-Username
<code>shader_name</code>	string	nein	Filtert nach Teil des Shader-Namens
<code>tags</code>	list[string]	nein	Aktuell vorbereitet, aber noch nicht fertig implementiert

Beispiel-Anfrage:

```
GET /1/shaders/filter/?shader_user_name=sebas&shader_name=Rainbow
```

Response 200:

```
[
  {
    "ShaderName": "Rainbow Diagonal",
    "ShaderCode": "#version 330 core\n...",
    "user_id": 2,
    "ShaderId": 1,
    "ShaderAuthor": "sebas",
    "ShaderTags": ["retro", "abstract"],
    "ShaderLikes": {
      "amount": 2,
      "liked_by_u": false
    },
    "ShaderTextures": []
  }
]
```

Fehler:

Code	Grund
404	<code>shader_user_name</code> wurde nicht gefunden

POST `/1/{user_id}/shaders/new`

Erstellt einen neuen Shader mit Standard-Shadercode. Im Body wird nur der Name übergeben.

URL-Parameter:

Parameter	Typ	Pflicht	Beschreibung
user_id	int	ja	User, dem der Shader gehört

Request-Body:

```
{
  "ShaderName": "Mein neuer Shader"
}
```

Feld	Typ	Pflicht	Beschreibung
ShaderName	string	ja	Name des neuen Shaders

Response 200:

```
{
  "ShaderName": "Mein neuer Shader",
  "ShaderCode": "#version 330 core\n\nout vec4 outputColor;\n...",
  "user_id": 1,
  "ShaderId": 4,
  "ShaderAuthor": "MaxMustermann",
  "ShaderTags": [],
  "ShaderLikes": {
    "amount": 0,
    "liked_by_u": false
  },
  "ShaderTextures": [],
  "ShaderComments": []
}
```

PUT /{user_id}/shaders/{shader_id}

Aktualisiert den Namen, Code und die Texturen eines Shaders.
Nur der Ersteller des Shaders darf diesen Shader bearbeiten.

URL-Parameter:

Parameter	Typ	Pflicht	Beschreibung
user_id	int	ja	ID des Users, der bearbeiten will

Parameter	Typ	Pflicht	Beschreibung
shader_id	int	ja	ID des Shaders

Request-Body:

```
{
  "ShaderName": "Neuer Shadername",
  "ShaderCode": "#version 330 core\n...",
  "user_id": 1,
  "ShaderTextures": [
    {
      "id": 1,
      "Texture64": "base64..."
    }
  ]
}
```

Feld	Typ	Pflicht	Beschreibung
ShaderName	string	ja	Neuer Name
ShaderCode	string	ja	Neuer GLSL-Code
user_id	int	ja	ID des Shader-Autors
ShaderTextures	list	ja	Liste der Texturen

Wichtig: Beim Update werden zuerst alle bisherigen Texturen dieses Shaders gelöscht. Danach werden die Texturen aus `ShaderTextures` neu gespeichert.

Response 200:

```
{
  "ShaderName": "Neuer Shadername",
  "ShaderCode": "#version 330 core\n...",
  "user_id": 1
}
```

Fehler:

Code	Grund
404	Shader wurde nicht gefunden
403	Nur der Ersteller darf den Shader bearbeiten

Kurzübersicht Shader

Methode	Pfad	Beschreibung
GET	<code>/ {user_id} / shaders /</code>	Alle Shader abrufen
GET	<code>/ {user_id} / shaders / {shader_id}</code>	Einzelnen Shader abrufen
GET	<code>/ {user_id} / shaders / filter /</code>	Shader filtern
POST	<code>/ {user_id} / shaders / new</code>	Neuen Shader mit Standardcode erstellen
PUT	<code>/ {user_id} / shaders / {shader_id}</code>	Shader bearbeiten

ShaderTag-API Dokumentation

Basis-URL: `http(s)://<server>/{user_id}/shaders/shadertag`

Authentifizierung: Im aktuellen Router ist kein API-Key aktiv.

GET `/ {user_id} / shaders / shadertag / {shader_id}`

Gibt alle Tags eines bestimmten Shaders als String-Liste zurück.

URL-Parameter:

Parameter	Typ	Pflicht	Beschreibung
<code>user_id</code>	int	ja	User-ID im Pfad
<code>shader_id</code>	int	ja	ID des Shaders

Beispiel-Anfrage:

```
GET /1/shaders/shadertag/3
```

Response 200:

```
[
  "abstract",
  "raymarching"
]
```


POST /{user_id}/shaders/shadertag/{shader_id}/{tag_id}

Verknüpft einen vorhandenen Tag mit einem Shader.

Nur der Autor des Shaders darf Tags zum Shader hinzufügen.

URL-Parameter:

Parameter	Typ	Pflicht	Beschreibung
user_id	int	ja	User, der die Änderung ausführt
shader_id	int	ja	Shader, der den Tag bekommen soll
tag_id	int	ja	Tag, der hinzugefügt wird

Beispiel-Anfrage:

```
POST /1/shaders/shadertag/4/2
```

Response 200:

```
{
  "tag_id": 2,
  "shader_id": 4,
  "user_id": 1
}
```

Fehler:

Code	Grund
404	Shader wurde nicht gefunden
404	Tag wurde nicht gefunden
403	Nur der Autor darf ShaderTags ändern
400	Der Shader hat diesen Tag bereits

DELETE

/ {user_id}/shaders/shadertag/{shader_id}/{tag_name}

Entfernt einen Tag anhand seines Namens von einem Shader.

Nur der Autor des Shaders darf Tags entfernen.

URL-Parameter:

Parameter	Typ	Pflicht	Beschreibung
user_id	int	ja	User, der die Änderung ausführt
shader_id	int	ja	Shader, von dem der Tag entfernt wird
tag_name	string	ja	Name des Tags

Beispiel-Anfrage:

```
DELETE /1/shaders/shadertag/4/abstract
```

Response 200:

```
{
  "tag_id": 2,
  "shader_id": 4,
  "user_id": 1
}
```

Fehler:

Code	Grund
404	Shader wurde nicht gefunden
404	Tag wurde nicht gefunden
404	Shader besitzt diesen Tag nicht
403	Nur der Autor darf ShaderTags ändern

Kurzübersicht ShaderTags

Methode	Pfad	Beschreibung
GET	<code>/1/shaders/shadertag/4</code>	Tags eines Shaders abrufen
POST	<code>/1/shaders/shadertag/4/abstract</code>	Tag zu Shader hinzufügen
DELETE	<code>/1/shaders/shadertag/4/abstract</code>	Tag von Shader entfernen

ShaderTexture-API Dokumentation

Basis-URL: `http(s)://<server>/{user_id}/shaders/{shader_id}/shadertexture`

Authentifizierung: Im aktuellen Router ist kein API-Key aktiv.

GET `/ {user_id}/shaders/{shader_id}/shadertexture`

Gibt alle gespeicherten Texturen eines Shaders zurück.

URL-Parameter:

Parameter	Typ	Pflicht	Beschreibung
<code>user_id</code>	int	ja	User-ID im Pfad
<code>shader_id</code>	int	ja	ID des Shaders

Beispiel-Anfrage:

```
GET /1/shaders/3/shadertexture
```

Response 200:

```
[
  {
    "id": 1,
    "Texture64": "base64..."
  }
]
```

Fehler:

Code	Grund
404	Shader wurde nicht gefunden

POST `/ {user_id}/shaders/{shader_id}/shadertexture`

Speichert eine neue Base64-Textur zu einem Shader.

URL-Parameter:

Parameter	Typ	Pflicht	Beschreibung
user_id	int	ja	User-ID im Pfad
shader_id	int	ja	ID des Shaders

Query-Parameter:

Parameter	Typ	Pflicht	Beschreibung
Encoded	string	ja	Base64-codierte Textur

Beispiel-Anfrage:

```
POST /1/shaders/3/shadertexture?Encoded=base64...
```

Response 200:

```
{
  "id": 1,
  "Texture64": "base64...",
  "shader_id": 3
}
```

PUT

`/1/{user_id}/shaders/{shader_id}/shadertexture/{TextureId}`

Aktualisiert eine vorhandene Textur.

URL-Parameter:

Parameter	Typ	Pflicht	Beschreibung
user_id	int	ja	User-ID im Pfad
shader_id	int	ja	ID des Shaders
TextureId	int	ja	ID der Textur

Query-Parameter:

Parameter	Typ	Pflicht	Beschreibung
Encoded	string	ja	Neuer Base64-String

Beispiel-Anfrage:

```
PUT /1/shaders/3/shadertexture/1?Encoded=neuerBase64String
```

Response 200:

```
{
  "id": 1,
  "Texture64": "neuerBase64String",
  "shader_id": 3
}
```

Fehler:

Code	Grund
404	Textur wurde nicht gefunden

DELETE

/ {user_id} / shaders / {shader_id} / shadertexture / {TextureId}

Löscht eine Textur anhand ihrer ID.

URL-Parameter:

Parameter	Typ	Pflicht	Beschreibung
user_id	int	ja	User-ID im Pfad
shader_id	int	ja	ID des Shaders
TextureId	int	ja	ID der Textur

Beispiel-Anfrage:

```
DELETE /1/shaders/3/shadertexture/1
```

Response: 204 No Content

Kurzübersicht ShaderTextures

Methode	Pfad	Beschreibung
GET	<code>/ {user_id} / shaders / {shader_id} / shadertexture</code>	Texturen und Shader abrufen
POST	<code>/ {user_id} / shaders / {shader_id} / shadertexture</code>	Textur hinzufügen
PUT	<code>/ {user_id} / shaders / {shader_id} / shadertexture / {TextureId}</code>	Textur ändern
DELETE	<code>/ {user_id} / shaders / {shader_id} / shadertexture / {TextureId}</code>	Textur löschen

Tags-API Dokumentation

Basis-URL: `http(s)://<server>/tags`

Authentifizierung: Im aktuellen Router ist kein API-Key aktiv.

Wichtige Regeln vorab

Tags sind globale Einträge. Ein Tag wird nur einmal in der Tabelle `Tags` gespeichert und kann danach mit mehreren Shadern verknüpft werden.

`TagName` darf maximal 31 Zeichen lang sein und muss eindeutig sein.

GET /tags/

Gibt alle verfügbaren Tags zurück.

Beispiel-Anfrage:

```
GET /tags/
```

Response 200:

```
[
  {
    "TagName": "retro",
    "TagId": 1
  }
]
```

```
    },
    {
      "TagName": "abstract",
      "TagId": 2
    }
  ]
}
```

GET /tags/{tag_id}

Gibt einen bestimmten Tag anhand seiner ID zurück.

URL-Parameter:

Parameter	Typ	Pflicht	Beschreibung
tag_id	int	ja	ID des Tags

Beispiel-Anfrage:

```
GET /tags/2
```

Response 200:

```
{
  "TagName": "abstract",
  "TagId": 2
}
```

Fehler:

Code	Grund
404	Tag wurde nicht gefunden

POST /tags/

Erstellt einen neuen Tag.

Request-Body:

```
{
  "TagName": "noise"
}
```

Feld	Typ	Pflicht	Regeln
TagName	string	ja	maximal 31 Zeichen, eindeutig

Response 200:

```
{
  "TagName": "noise",
  "TagId": 5
}
```

Fehler:

Code	Grund
409	Tag existiert bereits

DELETE /tags/delete_id/{tag_id}

Löscht einen Tag anhand seiner ID.

URL-Parameter:

Parameter	Typ	Pflicht	Beschreibung
tag_id	int	ja	ID des Tags

Beispiel-Anfrage:

```
DELETE /tags/delete_id/2
```

Response: 204 No Content

Fehler:

Code	Grund
404	Tag wurde nicht gefunden

DELETE /tags/delete_name/{tag_name}

Löscht einen Tag anhand seines Namens.

URL-Parameter:

Parameter	Typ	Pflicht	Beschreibung
tag_name	string	ja	Name des Tags

Beispiel-Anfrage:

```
DELETE /tags/delete_name/abstract
```

Response: 204 No Content

Fehler:

Code	Grund
404	Tagname wurde nicht gefunden

Kurzübersicht Tags

Methode	Pfad	Beschreibung
GET	/tags/	Alle Tags abrufen
GET	/tags/{tag_id}	Tag per ID abrufen
POST	/tags/	Tag erstellen
DELETE	/tags/delete_id/{tag_id}	Tag per ID löschen
DELETE	/tags/delete_name/{tag_name}	Tag per Namen löschen

Likes-API Dokumentation

Basis-URL: http(s)://<server>/{user_id}/{shader_id}/likes

Authentifizierung: Im aktuellen Router ist kein API-Key aktiv.

Wichtige Regeln vorab

Ein Like verbindet einen User mit einem Shader.

Der `POST`-Endpunkt funktioniert wie ein Toggle: Wenn der Like noch nicht existiert, wird er erstellt. Wenn er bereits existiert, wird er entfernt.

GET `/{{user_id}}/{{shader_id}}/likes/`

Gibt zurück, wie viele Likes ein Shader hat und ob der aktuelle User diesen Shader geliked hat.

URL-Parameter:

Parameter	Typ	Pflicht	Beschreibung
<code>user_id</code>	int	ja	User, für den <code>liked_by_u</code> geprüft wird
<code>shader_id</code>	int	ja	Shader, dessen Likes geprüft werden

Beispiel-Anfrage:

```
GET /1/3/likes/
```

Response 200:

```
{
  "amount": 4,
  "liked_by_u": true
}
```

Fehler:

Code	Grund
400	<code>shader_id</code> existiert nicht
400	<code>user_id</code> existiert nicht

POST `/{{user_id}}/{{shader_id}}/likes/`

Erstellt oder entfernt einen Like.

URL-Parameter:

Parameter	Typ	Pflicht	Beschreibung
user_id	int	ja	User, der liked
shader_id	int	ja	Shader, der geliked wird

Beispiel-Anfrage:

```
POST /1/3/likes/
```

Response 200, wenn Like neu erstellt wurde:

```
{
  "user_id": 1,
  "shader_id": 3
}
```

Response 200, wenn Like bereits existierte und entfernt wurde:

```
{
  "user_id": 0,
  "shader_id": 0
}
```

Fehler:

Code	Grund
400	shader_id existiert nicht
400	user_id existiert nicht

DELETE /{user_id}/{shader_id}/likes/

Entfernt einen Like direkt.

URL-Parameter:

Parameter	Typ	Pflicht	Beschreibung
user_id	int	ja	User, dessen Like entfernt wird
shader_id	int	ja	Shader, bei dem der Like entfernt wird

Beispiel-Anfrage:

```
DELETE /1/3/likes/
```

Response 200:

```
{
  "LikeId": 1,
  "shader_id": 3,
  "user_id": 1
}
```

Fehler:

Code	Grund
400	<code>shader_id</code> existiert nicht
400	<code>user_id</code> existiert nicht

Kurzübersicht Likes

Methode	Pfad	Beschreibung
GET	<code>/1/3/likes/</code>	Likes eines Shaders abrufen
POST	<code>/1/3/likes/</code>	Like toggeln
DELETE	<code>/1/3/likes/</code>	Like löschen

Comments-API Dokumentation

Basis-URL: `http(s)://<server>/1/3/comments`

Authentifizierung: Im aktuellen Router ist kein API-Key aktiv.

Wichtige Regeln vorab

Kommentare gehören zu einem Shader und einem User.

Beim Abrufen wird der Name des Autors aus der User-Tabelle gelesen.

GET /{user_id}/{shader_id}/comments/

Gibt alle Kommentare eines Shaders zurück.

URL-Parameter:

Parameter	Typ	Pflicht	Beschreibung
user_id	int	ja	User-ID im Pfad
shader_id	int	ja	Shader, dessen Kommentare geladen werden

Beispiel-Anfrage:

```
GET /1/3/comments/
```

Response 200:

```
[
  {
    "CommentAuthor": "sebas",
    "CommentText": "Cool inversion effect."
  }
]
```

Fehler:

Code	Grund
404	Shader wurde nicht gefunden

POST /{user_id}/{shader_id}/comments/

Erstellt einen neuen Kommentar zu einem Shader.

URL-Parameter:

Parameter	Typ	Pflicht	Beschreibung
user_id	int	ja	Autor des Kommentars
shader_id	int	ja	Shader, zu dem kommentiert wird

Request-Body:

```
{
  "CommentText": "Sehr cooler Shader!"
}
```

Feld	Typ	Pflicht	Regeln
CommentText	string	ja	maximal 511 Zeichen

Response 200:

```
{
  "CommentText": "Sehr cooler Shader!",
  "user_id": 1,
  "shader_id": 3
}
```

Fehler:

Code	Grund
404	Shader wurde nicht gefunden
404	User wurde nicht gefunden

DELETE /{user_id}/{shader_id}/comments/{comment_id}

Löscht einen Kommentar anhand seiner ID.

URL-Parameter:

Parameter	Typ	Pflicht	Beschreibung
user_id	int	ja	User-ID im Pfad
shader_id	int	ja	Shader-ID im Pfad
comment_id	int	ja	ID des Kommentars

Beispiel-Anfrage:

```
DELETE /1/3/comments/7
```

Response 200: leerer Body oder gelöschter Eintrag, abhängig von FastAPI-Ausgabe.

Fehler:

Code	Grund
400	Kommentar existiert nicht

Kurzübersicht Comments

Methode	Pfad	Beschreibung
GET	<code>/ {user_id} / {shader_id} / comments /</code>	Kommentare eines Shaders abrufen
POST	<code>/ {user_id} / {shader_id} / comments /</code>	Kommentar erstellen
DELETE	<code>/ {user_id} / {shader_id} / comments / {comment_id}</code>	Kommentar löschen

Datenbankmodell

Die API verwendet SQLite mit SQLAlchemy. Die Datenbankdatei heißt:

```
meine_app.db
```

Tabellen

Tabelle	Zweck
Users	Speichert Benutzer und Admins
Shaders	Speichert Shader-Namen und GLSL-Code
Comments	Speichert Kommentare zu Shadern
Tags	Speichert globale Tags
ShaderTags	Verknüpft Shader mit Tags
Likes	Verknüpft User mit gelikten Shadern
Textures	Speichert Base64-Texturen zu Shadern
Logging	Speichert Log-Einträge pro User

Users

Spalte	Typ	Bedeutung
UserId	int	Primärschlüssel
UserName	string(31)	Eindeutiger Benutzername
passwd	string	Passwort oder Passwort-Hash
is_admin	boolean	Gibt an, ob der User Admin ist

Shaders

Spalte	Typ	Bedeutung
ShaderId	int	Primärschlüssel
ShaderName	string(63)	Name des Shaders
ShaderCode	string	GLSL-Code
user_id	int	Fremdschlüssel auf Users.UserId

Comments

Spalte	Typ	Bedeutung
CommentId	int	Primärschlüssel
CommentText	string	Kommentartext
CommentAuthor	string	Autorname
user_id	int	Fremdschlüssel auf Users.UserId
shader_id	int	Fremdschlüssel auf Shaders.ShaderId

Tags

Spalte	Typ	Bedeutung
TagId	int	Primärschlüssel
TagName	string(31)	Eindeutiger Tagname

ShaderTags

Spalte	Typ	Bedeutung
ShaderTagsID	int	Primärschlüssel
shader_id	int	Fremdschlüssel auf Shaders.ShaderId

Spalte	Typ	Bedeutung
tag_id	int	Fremdschlüssel auf Tags.TagId
user_id	int	User, der die Verknüpfung erstellt hat

Likes

Spalte	Typ	Bedeutung
LikeId	int	Primärschlüssel
shader_id	int	Fremdschlüssel auf Shaders.ShaderId
user_id	int	Fremdschlüssel auf Users.UserId

Textures

Spalte	Typ	Bedeutung
id	int	Primärschlüssel
Texture64	string	Base64-codierte Textur
shader_id	int	Fremdschlüssel auf Shaders.ShaderId

Logging

Spalte	Typ	Bedeutung
id	int	Primärschlüssel
user_id	int	Fremdschlüssel auf Users.UserId
LogText	string	Text des Log-Eintrags

Gesamtübersicht aller Endpunkte

Methode	Pfad	Bereich
GET	/	Root
GET	/user/{username}	User
POST	/user/login	User
POST	/user/	User
PUT	/user/	User
DELETE	/user/	User
POST	/user/Log	User

Methode	Pfad	Bereich
GET	/admin/Log	Admin
DELETE	/admin/Log/{user_id}	Admin
GET	/admin/	Admin
GET	/admin/{username}	Admin
POST	/admin/login	Admin
POST	/admin/	Admin
PUT	/admin/	Admin
DELETE	/admin/	Admin
GET	/ {user_id}/shaders/	Shader
GET	/ {user_id}/shaders/{shader_id}	Shader
GET	/ {user_id}/shaders/filter/	Shader
POST	/ {user_id}/shaders/new	Shader
PUT	/ {user_id}/shaders/{shader_id}	Shader
GET	/ {user_id}/shaders/shadertag/{shader_id}	ShaderTa
POST	/ {user_id}/shaders/shadertag/{shader_id}/{tag_id}	ShaderTa
DELETE	/ {user_id}/shaders/shadertag/{shader_id}/{tag_name}	ShaderTa
GET	/ {user_id}/shaders/{shader_id}/shadertexture	ShaderTe
POST	/ {user_id}/shaders/{shader_id}/shadertexture	ShaderTe
PUT	/ {user_id}/shaders/{shader_id}/shadertexture/{TextureId}	ShaderTe
DELETE	/ {user_id}/shaders/{shader_id}/shadertexture/{TextureId}	ShaderTe
GET	/tags/	Tags
GET	/tags/{tag_id}	Tags
POST	/tags/	Tags
DELETE	/tags/delete_id/{tag_id}	Tags
DELETE	/tags/delete_name/{tag_name}	Tags
GET	/ {user_id}/{shader_id}/likes/	Likes
POST	/ {user_id}/{shader_id}/likes/	Likes
DELETE	/ {user_id}/{shader_id}/likes/	Likes
GET	/ {user_id}/{shader_id}/comments/	Comment
POST	/ {user_id}/{shader_id}/comments/	Comment
DELETE	/ {user_id}/{shader_id}/comments/{comment_id}	Comment